Goldilocks Field 상세 분석

소개

Goldilocks Field는 plonky2에서 사용하는 핵심 유한체(Finite Field)입니다. 이 문서는 Goldilocks Field의 수학적 정의, SNARK에 적합한 이유, 그리고 실제 구현 방법을 상세히 설명합니다.

1. 수학적 정의

1.1 기본 정의

Goldilocks Field는 다음과 같이 정의되는 소수 위수(prime order)를 가진 유한체입니다:

```
P = 2^64 - 2^32 + 1

= 18,446,744,069,414,584,321

= 0xFFFFFFFF00000001 (16진수)
```

이 소수는 다음과 같이 분해할 수 있습니다:

```
P = 2^64 - EPSILON
```

여기서 `EPSILON = 2^32 - 1 = 0xFFFFFFFF` 입니다.

또는 다음과 같이 표현할 수도 있습니다:

```
P = 2^32 * (2^32 - 1) + 1
```

1.2 코드 정의

plonky2의 `goldilocks_field.rs` 에서 Goldilocks Field는 다음과 같이 정의됩니다:

```
const EPSILON: u64 = (1 << 32) - 1;


#[derive(Copy, Clone, Serialize, Deserialize)]

#[repr(transparent)]

pub  struct GoldilocksField(pub u64);


impl Field64 for GoldilocksField {

const ORDER: u64 = 0xFFFFFFFF00000001;

// ...

}
```

`GoldilocksField` 는 내부적으로 `u64` 값 하나만 저장하는 간단한 구조체입니다. `#[repr(transparent)]` 속성은 메모리 레이아웃이 `u64` 와 동일함을 보장합니다.

2. SNARK에 적합한 이유

Goldilocks Field가 SNARK 증명 시스템에 특히 적합한 이유는 다음과 같습니다:

2.1 64비트 워드 크기

- 현대 CPU의 네이티브 워드 크기가 64비트이므로, 단일 레지스터에 필드 원소를 저장할 수 있습니다
- 메모리 효율성이 뛰어나며, 캐시 친화적입니다

- 다른 필드(예: BN254의 254비트)에 비해 훨씬 빠른 연산이 가능합니다

2.2 높은 2-adicity

```
const TWO_ADICITY: usize = 32;
```

2-adicity가 32라는 것은 `P - 1 = 2^32 * k` (k는 홀수)로 표현된다는 의미입니다. 이는 다음과 같은 장점을 제공합니다:

- **FFT 효율성**: 최대 2^32 크기의 FFT를 효율적으로 수행할 수 있습니다
- **다항식 연산**: 다항식 보간과 평가가 매우 빠릅니다
- **회로 크기**: 최대 2^32개의 게이트를 가진 회로를 지원할 수 있습니다

실제로 2^32 = 4,294,967,296개의 게이트는 대부분의 실용적인 회로에 충분합니다.

2.3 빠른 모듈러 연산

`P = 2^64 - 2^32 + 1` 형태의 소수는 특별한 구조를 가지고 있어, 모듈러 리덕션(modular reduction)을 매우 빠르게 수행할 수 있습니다. 일반적인 나눗셈 연산 대신, 비트 시프트와 덧셈/뺄셈만으로 리덕션이 가능합니다.

2.4 재귀 증명 지원

Goldilocks Field는 다른 필드(예: BN254)와의 호환성을 고려하여 설계되었습니다. 이를 통해:

- Goldilocks Field에서 생성된 증명을 다른 필드에서 검증 가능
- 재귀 증명(recursive proof) 구성이 효율적
- 다양한 블록체인 환경과의 통합이 용이

3. 주요 연산 구현

3.1 덧셈 연산 (add)

덧셈은 오버플로우를 처리하면서 모듈러 연산을 수행합니다:

```
impl Add for GoldilocksField {
type Output = Self;

#[inline]
fn add(self, rhs: Self) -> Self {
let (sum, over) = self.0.overflowing_add(rhs.0);
let (mut sum, over) = sum.overflowing_add((over as u64) * EPSILON);
if over {
// 이중 오버플로우는 매우 드물지만, 발생 시 처리
assume(self.0 > Self::ORDER && rhs.0 > Self::ORDER);
branch_hint();
sum += EPSILON; // 오버플로우 불가
}
Self(sum)
}
}
```

동작 원리:

1. **첫 번째 덧셈**: `self.0 + rhs.0` 를 수행하고 오버플로우 여부를 확인
2. **첫 번째 조정**: 오버플로우가 발생했다면 `EPSILON` 을 더함

- 이는 `2^64` 를 더한 것을 `2^64 - EPSILON` 으로 나눈 나머지를 구하는 것과 같음
- `2^64 ≡ EPSILON (mod P)` 이므로 `EPSILON` 을 더하면 됨

3. **두 번째 조정**: 드물게 이중 오버플로우가 발생하면 다시 `EPSILON` 을 더함

예시:

```
// 예시 1: 일반적인 경우 (오버플로우 없음)
let a = GoldilocksField(100);
let b = GoldilocksField(200);
let c = a + b; // c.0 = 300

// 예시 2: 오버플로우 발생
let a = GoldilocksField(0xFFFFFFFF00000000); // P - 1에 가까운 값
let b = GoldilocksField(0x0000000100000000);
// 첫 번째 덧셈: 오버플로우 발생
// 조정: EPSILON 추가
```

3.2 뺄셈 연산 (sub)

뺄셈은 언더플로우를 처리합니다:

```
impl Sub for GoldilocksField {
type Output = Self;

#[inline]
fn sub(self, rhs: Self) -> Self {
let (diff, under) = self.0.overflowing_sub(rhs.0);
let (mut diff, under) = diff.overflowing_sub((under as u64) * EPSILON);
```

```
if under {
// 이중 언더플로우는 매우 드물지만, 발생 시 처리
assume(self.0 < EPSILON - 1 && rhs.0 > Self::ORDER);
branch_hint();
diff -= EPSILON; // 언더플로우 불가
}
Self(diff)
}
}
```

동작 원리:

1. **첫 번째 뺄셈**: `self.0 - rhs.0` 를 수행하고 언더플로우 여부를 확인
2. **첫 번째 조정**: 언더플로우가 발생했다면 `EPSILON` 을 뺌
3. **두 번째 조정**: 드물게 이중 언더플로우가 발생하면 다시 `EPSILON` 을 뺌

3.3 곱셈 연산 (mul)

곱셈은 128비트 중간 결과를 사용하여 정확한 계산을 수행합니다:

```
impl Mul for GoldilocksField {
type Output = Self;

#[inline]
fn mul(self, rhs: Self) -> Self {
reduce128((self.0 as u128) * (rhs.0 as u128))
```

```
}
}
```

곱셈의 핵심은 `reduce128` 함수입니다:

```
#[inline]
fn reduce128(x: u128) -> GoldilocksField {
let (x_lo, x_hi) = split(x); // x를 하위 64비트와 상위 64비트로 분리
let x_hi_hi = x_hi >> 32;
let x_hi_lo = x_hi & EPSILON;

let (mut t0, borrow) = x_lo.overflowing_sub(x_hi_hi);
if borrow {
branch_hint(); // borrow는 매우 드물게 발생
t0 -= EPSILON; // 언더플로우 불가
}
let t1 = x_hi_lo * EPSILON;
let t2 = unsafe { add_no_canonicalize_trashing_input(t0, t1) };
GoldilocksField(t2)
}

#[inline]
const fn split(x: u128) -> (u64, u64) {
```

```
(x  as u64, (x >> 64) as u64)
}
```

동작 원리:

128비트 값 `x = x_lo + x_hi * 2^64` 를 필드 원소로 리덕션합니다.

1. **분해**: `x` 를 `x_lo` (하위 64비트)와 `x_hi` (상위 64비트)로 분리
2. **x_hi 분해**: `x_hi` 를 다시 `x_hi_hi` (상위 32비트)와 `x_hi_lo` (하위 32비트)로 분리
3. **리덕션 공식 적용**:

```
x = x_lo + x_hi * 2^64
= x_lo + (x_hi_hi * 2^32 + x_hi_lo) * 2^64
= x_lo + x_hi_hi * 2^96 + x_hi_lo * 2^64
```

`2^64 ≡ EPSILON (mod P)` 이고 `2^96 ≡ EPSILON * 2^32 ≡ -1 (mod P)` 이므로:

```
x ≡ x_lo - x_hi_hi + x_hi_lo * EPSILON (mod P)
```

4. **계산**: `t0 = x_lo - x_hi_hi`, `t1 = x_hi_lo * EPSILON`, `t2 = t0 + t1`

예시:

```
// 예시: 큰 수의 곱셈
let  a = GoldilocksField(0xFFFFFFFF00000000);
let  b = GoldilocksField(2);
let  c = a * b;
// 중간 결과: 0x1FFFFFFFE00000000 (128비트)
```

```
// reduce128을 통해 필드 원소로 변환
```

3.4 역원 계산 (try_inverse)

역원은 Fermat의 소정리(Fermat's Little Theorem)를 사용하여 계산됩니다:

```
fn try_inverse(&self) -> Option<Self> {
if self.is_zero() {
return None;
}

// P - 2를 지수로 하여 거듭제곱 계산
// a^(P-2) = a^(-1) (mod P)
// P - 2의 이진 표현:
// 0b1111111111111111111111111111111011111111111111111111111111111111
// base^11 계산
let t2 = self.square() * *self;

// base^111 계산
let t3 = t2.square() * *self;

// base^111111 계산 (6개의 1)
let t6 = exp_acc::<3>(t3, t3);
```

```
// base^111111111111 계산 (12개의 1)
let  t12 = exp_acc::<6>(t6, t6);


// base^111111111111111111111111 계산 (24개의 1)
let  t24 = exp_acc::<12>(t12, t12);


// base^1111111111111111111111111111111 계산 (31개의 1)
let  t30 = exp_acc::<6>(t24, t6);
let  t31 = t30.square() * *self;


// base^11111111111111111111111111111110111111111111111111111111111111111 계산
let  t63 = exp_acc::<32>(t31, t31);


// base^111111111111111111111111111111101111111111111111111111111111111111 계산
Some(t63.square() * *self)
}
```

수학적 배경:

Fermat의 소정리에 따르면, 소수 `p` 와 `0 < a < p` 에 대해:

```
a^(p-1) ≡ 1 (mod p)
```

따라서:

```
a^(p-2) * a ≡ 1 (mod p)
```

```
a^(p-2) ≡ a^(-1) (mod p)
```

즉, `a^(p-2)` 가 `a` 의 역원입니다.

최적화 전략:

`P - 2` 를 이진수로 표현하면:

```
P - 2 = 0b1111111111111111111111111111111011111111111111111111111111111111
```

이는 31개의 1, 1개의 0, 32개의 1로 구성됩니다. 이 구조를 활용하여:

1. 작은 거듭제곱을 먼저 계산 (`t2` , `t3` , `t6` , `t12` , `t24` , `t31`)
2. 이들을 조합하여 큰 거듭제곱을 효율적으로 계산
3. 총 72번의 곱셈으로 역원을 계산 (naive한 방법보다 훨씬 빠름)

보조 함수:

```
#[inline(always)]
fn  exp_acc<const N: usize>(base: GoldilocksField, tail: GoldilocksField) -> Gol
base.exp_power_of_2(N) * tail
}
```

이 함수는 `base` 를 `N` 번 제곱한 후 `tail` 을 곱합니다. 예를 들어:

- `exp_acc::<3>(t3, t3) = t3^(2^3) * t3 = t3^8 * t3 = t3^9`

예시:

```
// 예시: 역원 계산
```

```
let a = GoldilocksField(7);
let a_inv = a.try_inverse().unwrap();
let product = a * a_inv;
assert_eq!(product, GoldilocksField::ONE);
```

4. x86-64 최적화 기법

4.1 add_no_canonicalize_trashing_input

이 함수는 x86-64 아키텍처에서 덧셈을 최적화합니다:

```
#[inline(always)]
#[cfg(target_arch = "x86_64")]
unsafe fn add_no_canonicalize_trashing_input(x: u64, y: u64) -> u64 {
let res_wrapped: u64;
let adjustment: u64;
core::arch::asm!(
"add {0}, {1}",
// Carry 플래그가 설정되면 오버플로우 발생
// sbb x, y는 x := x - y - CF를 수행
// 여기서는 x와 y가 모두 {1:e}이므로, 오버플로우 시 0xffffffff, 아니면 0
"sbb {1:e}, {1:e}",
inlateout(reg) x => res_wrapped,
inlateout(reg) y => adjustment,
```

```
options(pure, nomem, nostack),
);
// EPSILON 추가 == ORDER 빼기
res_wrapped + adjustment
}
```

동작 원리:

1. **add 명령**: `x + y` 를 수행하고 결과를 `res_wrapped` 에 저장

- 오버플로우 발생 시 Carry 플래그(CF) 설정

2. **sbb 명령**: `sbb {1:e}, {1:e}` 는 자기 자신에서 자신과 CF를 빼는 연산

- CF = 1이면: `0 - 0 - 1 = 0xFFFFFFFF` (32비트 레지스터에서)
- CF = 0이면: `0 - 0 - 0 = 0`
- 결과적으로 `adjustment` 는 오버플로우 시 `EPSILON` , 아니면 `0`

3. **최종 덧셈**: `res_wrapped + adjustment`

장점:

- 분기(branch) 없이 조건부 덧셈 수행
- CPU 파이프라인 효율성 향상
- 캐시 미스 감소

주의사항:

- `unsafe` 함수: `x + y < 2^64 + ORDER` 를 가정
- 입력 레지스터를 덮어씀 (trashing input)
- 입력을 재사용하지 않을 때만 사용 권장

4.2 비정규형(Non-canonical) 표현

Goldilocks Field는 성능을 위해 비정규형 표현을 허용합니다:

```
#[inline(always)]
fn to_noncanonical_u64(&self) -> u64 {
self.0
}

#[inline]
fn to_canonical_u64(&self) -> u64 {
let mut c = self.0;
// 한 번의 조건부 뺄셈만 필요 (2 * ORDER는 u64에 맞지 않음)
if c >= Self::ORDER {
c -= Self::ORDER;
}
c
}
```

정규형 vs 비정규형:

- **정규형(Canonical)**: `0 ≤ value < ORDER`
- **비정규형(Non-canonical)**: `0 ≤ value < 2^64`

비정규형을 허용하면:

- 중간 계산에서 정규화를 생략하여 성능 향상
- 최종 결과나 비교 시에만 정규화 수행
- 메모리와 연산 오버헤드 감소

예시:

```
// 비정규형 허용
let  a = GoldilocksField(0xFFFFFFFF00000002); // ORDER + 1
let  b = GoldilocksField(1);
// a와 b는 수학적으로 같지만, 내부 표현이 다름


// 정규화 후 비교
assert_eq!(a.to_canonical_u64(), b.to_canonical_u64());
```

5. 성능 특성

5.1 연산 복잡도

| 연산 | 복잡도 | 설명 |

|------|--------|------|

| 덧셈 | O(1) | 2-3개의 64비트 덧셈 |

| 뺄셈 | O(1) | 2-3개의 64비트 뺄셈 |

| 곱셈 | O(1) | 1개의 128비트 곱셈 + 리덕션 |

| 역원 | O(log P) | 72번의 곱셈 (Fermat) |

| 정규화 | O(1) | 1번의 비교 + 조건부 뺄셈 |

5.2 다른 필드와의 비교

| 필드 | 크기 | 덧셈 속도 | 곱셈 속도 | 2-adicity |

|------|------|-----------|-----------|-----------|

| Goldilocks | 64비트 | 매우 빠름 | 매우 빠름 | 32 |

| BN254 | 254비트 | 느림 | 느림 | 1 |

| BLS12-381 | 381비트 | 매우 느림 | 매우 느림 | 32 |

Goldilocks Field는:

- BN254보다 약 5-10배 빠름
- BLS12-381보다 약 10-20배 빠름
- 메모리 사용량이 1/4 ~ 1/6 수준

5.3 실제 성능 측정

plonky2의 벤치마크 결과 (대략적인 수치):

```
덧셈: ~0.5 ns/op
곱셈: ~1.5 ns/op
역원: ~100 ns/op
FFT(2^20): ~50 ms
```

이는 현대 CPU에서 매우 빠른 성능입니다.

6. 사용 예시

6.1 기본 연산

```
use plonky2_field::goldilocks_field::GoldilocksField;
use plonky2_field::types::Field;
```

```rust
// 필드 원소 생성
let a = GoldilocksField::from_canonical_u64(42);
let b = GoldilocksField::from_canonical_u64(17);

// 덧셈
let sum = a + b;
assert_eq!(sum.to_canonical_u64(), 59);

// 곱셈
let product = a * b;
assert_eq!(product.to_canonical_u64(), 714);

// 역원
let a_inv = a.inverse();
let one = a * a_inv;
assert_eq!(one, GoldilocksField::ONE);

// 거듭제곱
let a_squared = a.square();
let a_cubed = a_squared * a;
```

6.2 다항식 연산

```rust
use plonky2_field::polynomial::PolynomialValues;


// 다항식 값 생성
let values: Vec<GoldilocksField> = (0..8)
.map(|i| GoldilocksField::from_canonical_u64(i))
.collect();
let poly = PolynomialValues::new(values);


// FFT 수행
let coeffs = poly.ifft();


// 다항식 평가
let x = GoldilocksField::from_canonical_u64(5);
let y = coeffs.eval(x);
```

6.3 회로에서의 사용

```rust
use plonky2::plonk::circuit_builder::CircuitBuilder;
use plonky2::plonk::config::{GenericConfig, PoseidonGoldilocksConfig};


const D: usize = 2;
type C = PoseidonGoldilocksConfig;
type F = <C as GenericConfig<D>>::F; // GoldilocksField
```

```
let  config = CircuitConfig::standard_recursion_config();
let  mut  builder = CircuitBuilder::<F, D>::new(config);


// Target 추가 (내부적으로 GoldilocksField 사용)
let  a = builder.add_virtual_target();
let  b = builder.add_virtual_target();
let  c = builder.mul(a, b);


// 모든 연산이 Goldilocks Field에서 수행됨
```

7. 확장체 (Extension Field)

Goldilocks Field는 확장체를 지원하여 더 복잡한 연산을 가능하게 합니다:

7.1 2차 확장체

```
// GoldilocksField의 2차 확장체
type F2 = QuadraticExtension<GoldilocksField>;


// 원소 생성: a + b * X (X^2 = 7)
let  elem = F2::from_basefield_array([
GoldilocksField::from_canonical_u64(3),
GoldilocksField::from_canonical_u64(4),
```

```
]);
```

확장체는 다음과 같은 경우에 사용됩니다:

- FRI 프로토콜의 폴딩(folding) 과정
- 재귀 증명에서 더 큰 보안 수준 제공
- 특정 암호학적 프로토콜 구현

8. 요약

Goldilocks Field는 plonky2의 핵심 구성 요소로, 다음과 같은 특징을 가집니다:

수학적 특성:

- 소수 위수: `P = 2^64 - 2^32 + 1`
- 높은 2-adicity (32)
- 64비트 표현

성능 특성:

- 매우 빠른 산술 연산
- 효율적인 FFT 지원
- 하드웨어 최적화 (x86-64, AVX2, AVX-512)

구현 특징:

- 비정규형 표현 허용
- Fermat의 소정리를 이용한 역원 계산
- 인라인 어셈블리를 통한 최적화

SNARK 적합성:

- 큰 회로 지원 (최대 2^32 게이트)

- 재귀 증명 효율성
- 다른 필드와의 호환성

이러한 특성들이 결합되어 plonky2를 세계에서 가장 빠른 SNARK 증명 시스템 중 하나로 만듭니다.

참고 자료

- plonky2 GitHub Repository
- Goldilocks Field 구현 코드
- 관련 문서: 01-architecture.md - 전체 아키텍처 개요
- 관련 문서: 02-circuit-building.md - 회로 구성에서의 사용
- 관련 문서: 05-fri-protocol.md - FRI에서의 활용

문서 네비게이션

- **이전**: 05. FRI 프로토콜 - FRI 프로토콜 상세
- **다음**: 07. Fibonacci 예제 - 기본 회로 구성 예제
- **관련 문서**:
- 01. 아키텍처 개요 - Field 모듈 구조
- 11. 성능 최적화 - 필드 연산 최적화
- 02. 회로 구성 프로세스 - 필드 사용 예제